

TRIBHUVAN UNIVERSITY · INSTITUTE OF ENGINEERING

PURWANCHAL CAMPUS, DHARAN



DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

A MINOR PROJECT PRESENTATION ON

DESIGN AND IMPLEMENTATION OF A TWO-WHEELED SELF-BALANCING ROBOT WITH OPTIMIZED PID CONTROL

PRESENTED BY

AADESH DAHAL

PUR080BEI002

NISHCHAL POKHREL

PUR080BEI027

PRASHAN CHENTA RAI

PUR080BEI030

YOGESH KHADKA

PUR080BEI048

AUGUST, 2026

Outline

- 1 Introduction
- 2 System Design
- 3 Implementation
- 4 Results
- 5 Conclusion

Introduction & Objectives

Background

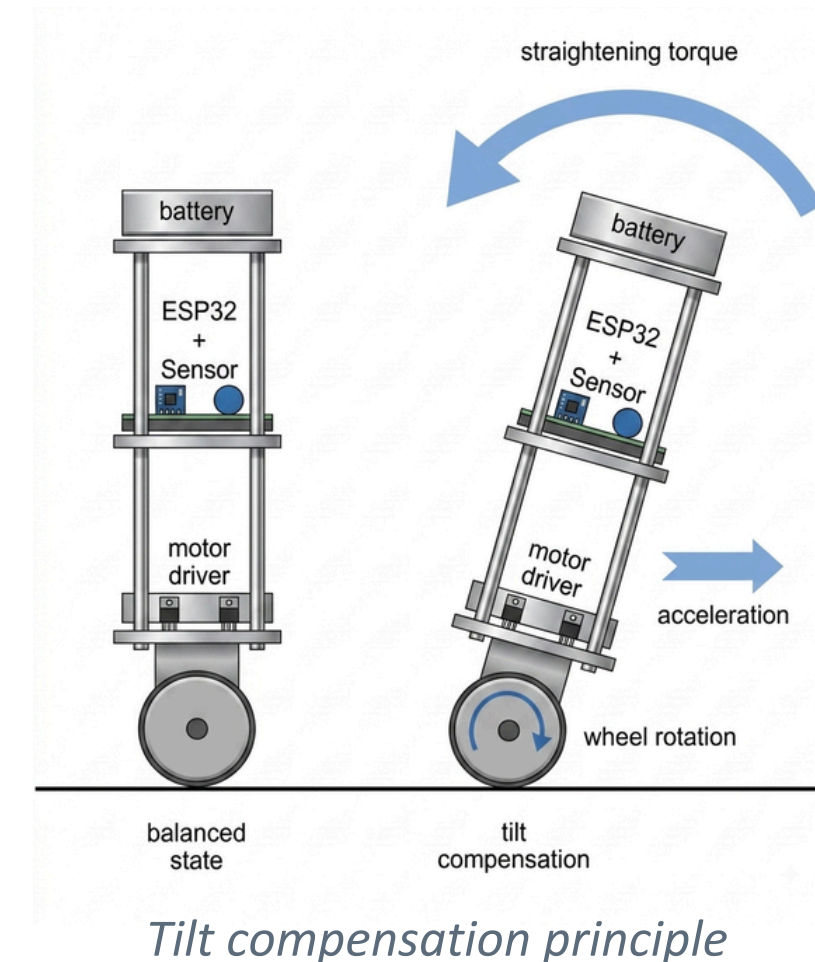
- Self-balancing robots keep an upright posture on two wheels using the inverted pendulum principle — inherently unstable without active control.
- Low-cost microcontrollers (ESP32) and MEMS IMUs (MPU6050) make real-time balancing feasible for academic projects.

Problem Statement

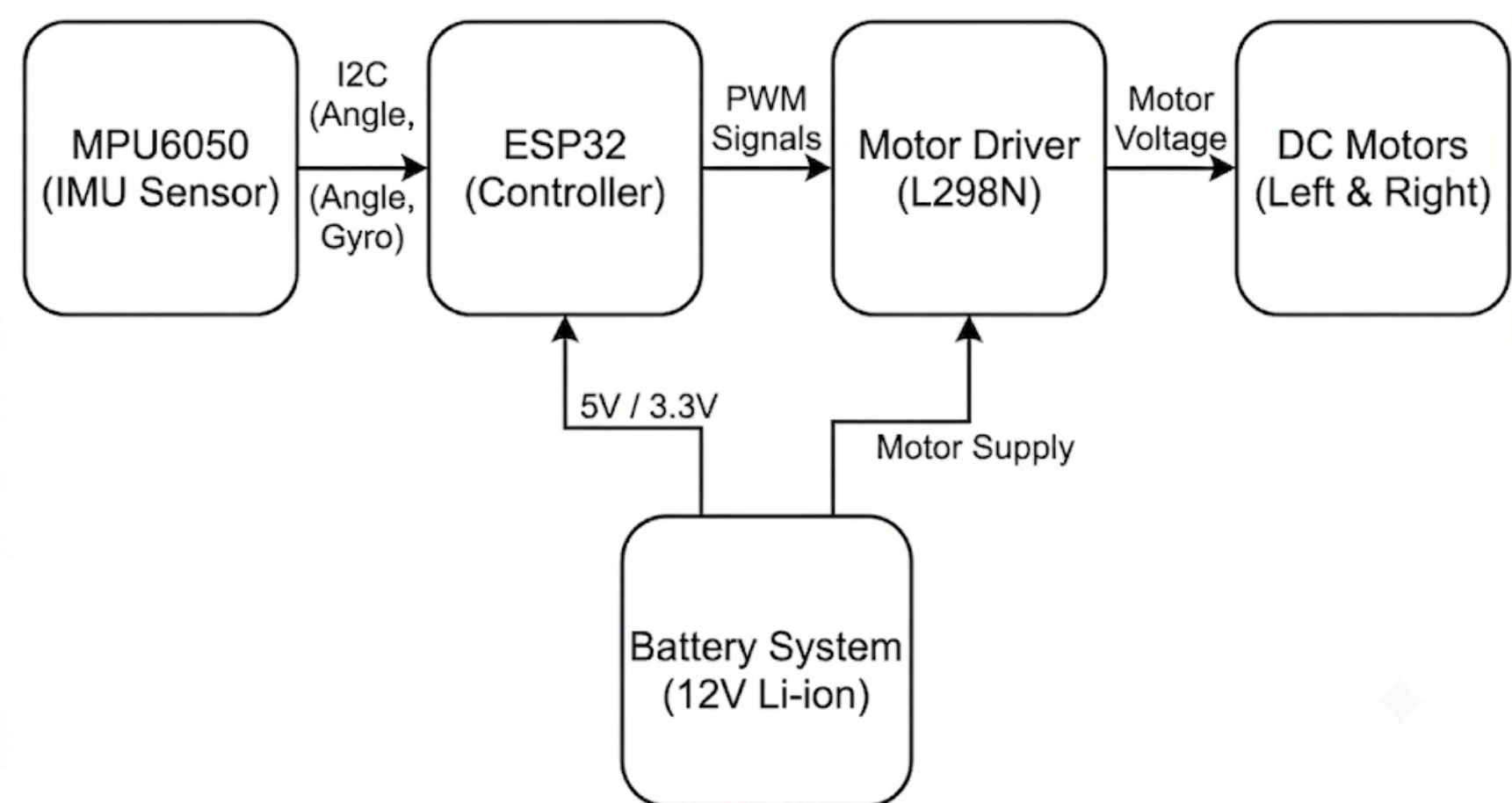
- Obtaining a reliable tilt angle from noisy IMU data.
- Designing a PID controller that handles motor nonlinearities and disturbances.
- Tuning PID gains for stable, responsive balancing.

Objectives

- Assemble the ESP32 + MPU6050 + motor-driver hardware.
- Implement a complementary filter for tilt estimation.
- Implement and tune a real-time PID balance controller.
- Evaluate stability and disturbance rejection.



System Architecture & Hardware

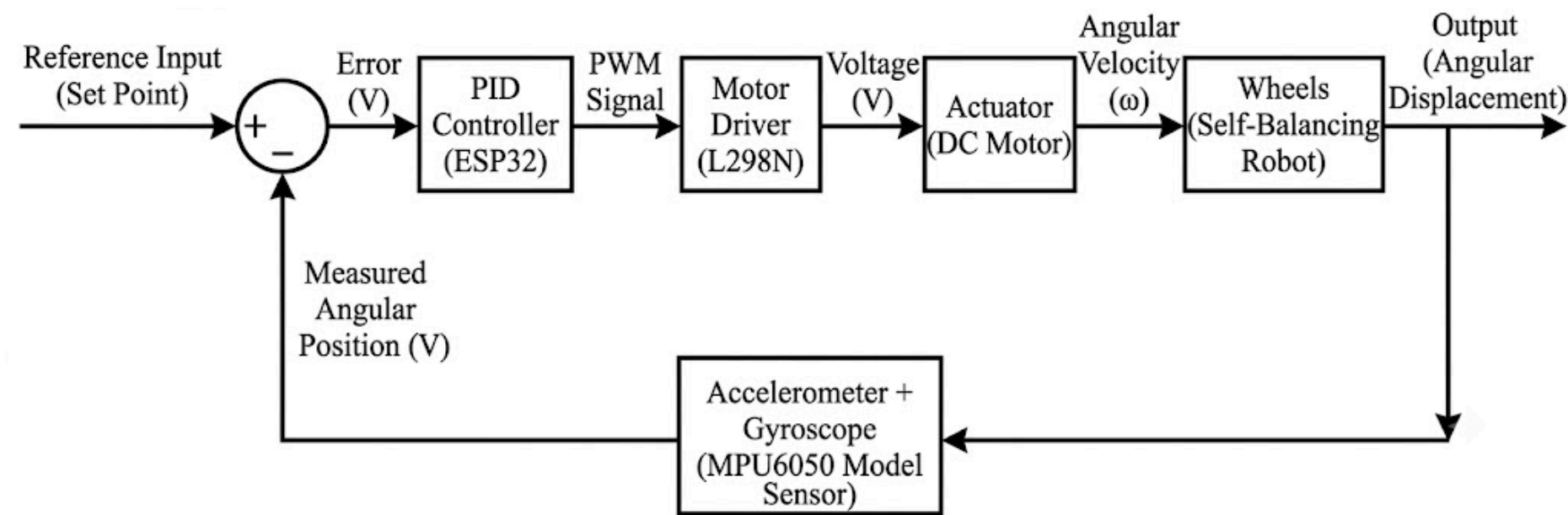


Block diagram of the self-balancing robot architecture

- MPU6050 measures tilt via I2C; ESP32 runs the control loop and outputs PWM; L298N drives the two geared DC motors.
- A 12V Li-ion pack, regulated by a buck converter, powers logic and motors.

Component	Function
ESP32 DevKit V1	Main controller: sensor fusion, PID, PWM generation
MPU6050 IMU	3-axis accelerometer + gyroscope for tilt sensing
L298N (x2)	Dual H-bridge motor driver
JGB37-520 DC motor	Geared motor (12V, 150 RPM) with quadrature encoder
3S Li-ion battery	12V primary power source
Buck converter	Regulates 5V / 3.3V rails

Working Principle: PID Control & Angle Estimation



Closed-loop control: setpoint $\rightarrow$ PID $\rightarrow$ PWM $\rightarrow$ motors $\rightarrow$ IMU feedback

Discrete PID law

$$u[k] = K_p e[k] + K_i \sum e[j] \Delta t + K_d \cdot (e[k] - e[k-1]) / \Delta t$$

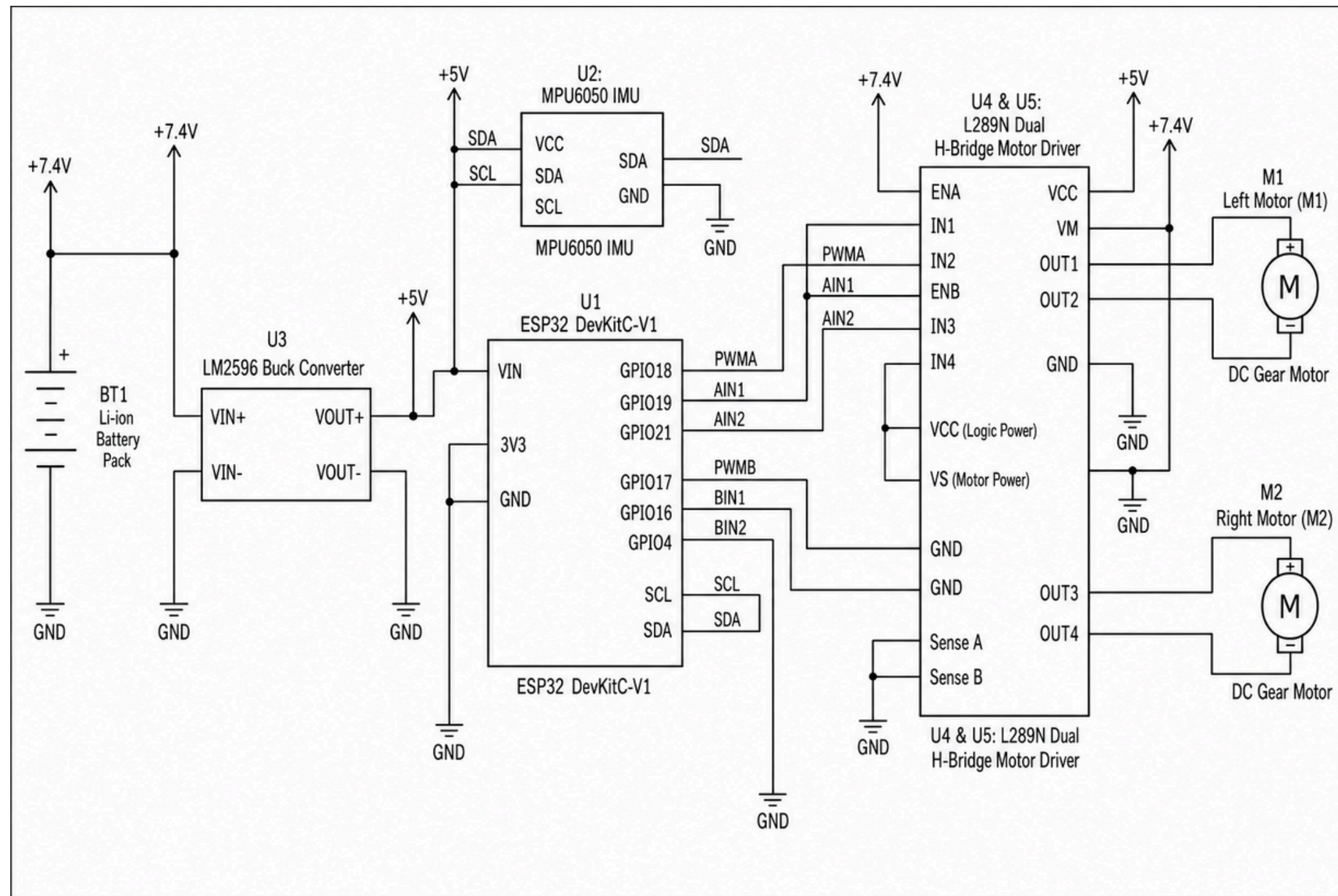
where $e[k] = \vartheta_{set} - \vartheta[k]$ is the tilt error.

Complementary filter (angle estimate)

$$\theta_{est}[k] = \alpha(\theta_{est}[k-1] + \omega_{gyro}[k]\Delta t) + (1-\alpha)\theta_{acc}[k]$$

- $\alpha \approx 0.98$ — low-passes the noisy accelerometer, high-passes the drifting gyroscope.
- Fuses both readings into a single stable tilt estimate for the PID loop.
- Control loop runs at a fixed 100 Hz (10 ms) sampling interval.
- Output $u[k]$ is a PWM duty cycle (–255 to 255) sent to the motor drivers; the derivative term uses gyro rate directly, avoiding noisy re-differentiation.

Hardware & Software Implementation



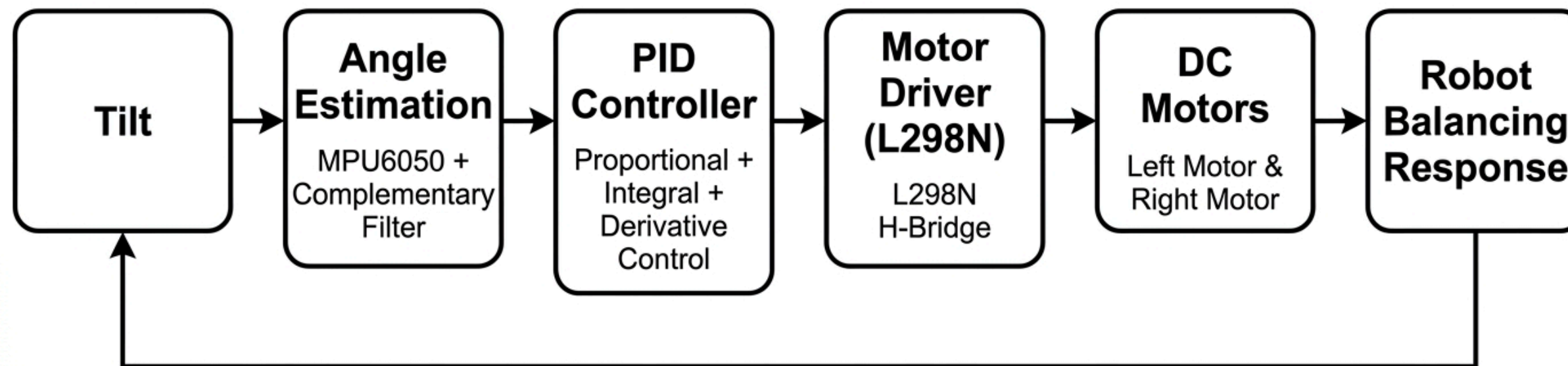
Complete circuit schematic: ESP32, MPU6050, dual L298N drivers, motors

Key software modules (100 Hz loop)

- MPU6050 calibration — gyro offset removal at startup
- Complementary-filter angle estimation
- Balance PID controller, output clamped to ± 255
- Left/right RPM synchronization from encoder feedback
- Automatic motor trim for persistent motor asymmetry
- Startup ramp (180 ms) and PWM slew-rate limiting (max change 10/cycle)
- Safety cut-off: motors stop if tilt $> 35^\circ$ or IMU fails repeatedly

Testing & Validation

- **MPU6050 calibration & angle test:** verified sign convention on forward/backward tilt
- **Complementary filter test:** checked estimate stability while stationary and tilting
- **PID balance test:** confirmed correction direction matched tilt direction
- **Encoder & RPM test:** validated wheel-speed feedback from quadrature encoders
- **Motor synchronization test:** verified left/right RPM correction reduces speed mismatch
- **Safety test:** confirmed motors stop on excessive tilt or repeated IMU failure
- **Final balancing test:** full closed-loop validation: tilt → angle estimate → PID → motors → response



Complete feedback sequence verified during final integration testing

Results: Balancing Performance

CURRENT PID GAINS

K_p

221.55

K_i

0.002

K_d

4.65

- Tuned gains: $K_p = 221.55$, $K_i = 0.002$, $K_d = 4.65$
- FreeRTOS control task holds a stable **113.2 Hz** loop rate, isolated from the WebSocket telemetry task (≈ 200 ms updates)
- Average error: -0.12°
- RMS error: 1.38°
- Peak error: 7.53°
- Sustained limit-cycle oscillation observed at **11.71 Hz**

Critical Analysis, Limitations & Future Scope

What the Diagnostics Revealed

- High $K_p = 221.55$ causes aggressive corrections.
- very low $K_i = 0.002$ provides little steady-state error compensation.
- $K_d = 4.65$ amplifies gyro noise, producing large D-term spikes.
- Pitch remains near setpoint but exhibits high-frequency oscillations.
- Lack of D-term filtering reduces stability and smoothness.

Current Gains & Future Work

Gains: $K_p = 221.55$, $K_i = 0.002$, $K_d = 4.65$

- Add a low-pass filter on gyro rate before the D-term
- Replace/extend the complementary filter with a Kalman filter
- Adaptive PID for varying battery voltage/load
- Add position/velocity control and autonomous navigation
- Higher-resolution data logging for frequency-domain analysis

Conclusion

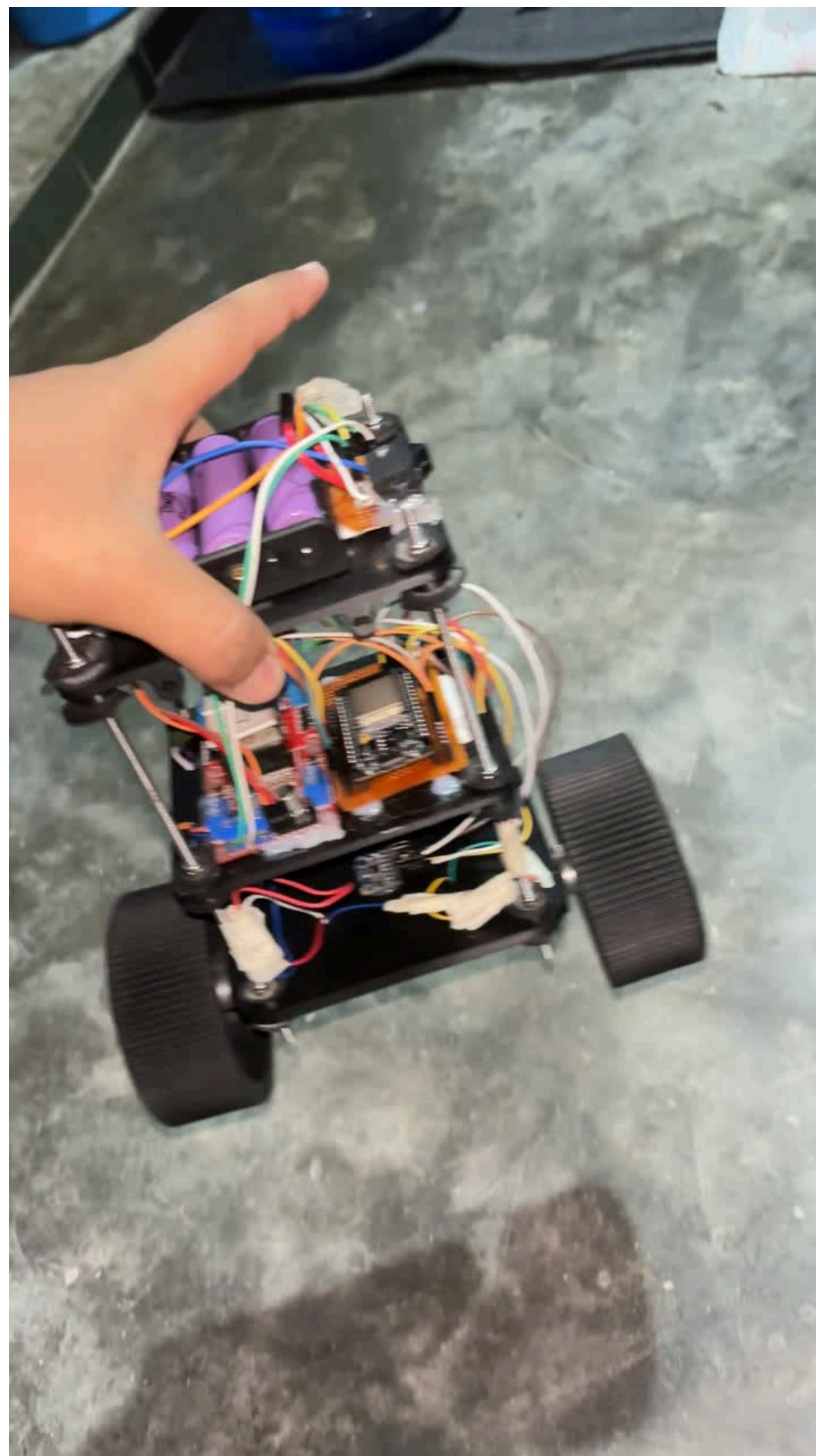
- Successfully built a two-wheeled self-balancing robot on **ESP32**, integrating **MPU6050** sensing, complementary filter angle estimation, and a real-time PID balance controller at **113.2 Hz**.
- FreeRTOS task separation and a WebSocket dashboard enabled robust live telemetry and PID tuning without disturbing the control loop.
- Achieved encoder-based RPM synchronization, automatic motor trim, startup ramping, PWM slew limiting, and tilt/IMU-failure safety cut-offs.
- Live telemetry diagnosed an **11.71 Hz** limit-cycle oscillation from an over-aggressive derivative gain and weak integral action — a clear, actionable finding rather than an unknown failure. Gains have since been updated to **$K_p = 221.55$, $K_i = 0.002$, $K_d = 4.65$** .
- The architecture is a robust foundation for the planned sensor-fusion and autonomous-navigation upgrades.



THANK YOU

We welcome your questions and suggestions.

Two-Wheeled Self-Balancing Robot



Demo Video